

Bangladesh Army University of Engineering & Technology (BAUET)
Qadirabad, Natore-6431
Department of Computer Science and Engineering

A Proposal for the Undergraduate Project/IDP

To be Entitled

Course Code: CSE 3200

Course Title: Integrated Development Project/ Capstone Project

SmartVoice: Personalized PC Assistant for Task Automation

Submitted By	Submitted To
Md Fahim Bin Alam, Md. Foisal Haque Sohom, Nishat Tasnim Chowdhury	Supervisor's Name: Mohammed Golam Sarwar Bhuyan
0812220205101037, 0812220205101057, 0812220105101022	Designation: Professor & Dean, Faculty of ECE
Batch: 16th	
Session: 2022-23	

Date: 11 September 2025

Approved: **Date:**

_____ _____

(Signature of the Thesis Supervisor)

Approved: **Date:**

_____ _____

(Signature of the Program Head)

Declaration

We hereby declare that the work presented in this proposal is our original work, and it has not been submitted elsewhere. All sources have been appropriately acknowledged.

Signature: _____

Name: _____

Date: _____

Table of Contents

Sl.	Description		Page no.
	Declaration		i
	Executive Summary		iii
1.	Introduction		1 - 4
	1.1	Problem Statement	1 - 2
	1.2	Objectives	2 - 3
	1.3	Scope and Limitations	3 - 4
2.	System Requirements		4 - 5
3.	Design Methodology		5 - 14
4.	System Architecture		14 - 15
5.	Feasibility & Risk Analysis		15 - 18
6.	Project Timeline		18
7.	Cost Estimation		19 - 20
8.	Expected Outcomes		20 - 21
9.	References		21
10.	Detailed Mapping of CO-PO-KP-CEP-CEA		22 - 25
11.	Supervisor Comments and Approval		25

Executive Summary

SmartVoice is an innovative desktop assistant that offers privacy-focused, offline task automation through voice commands, processing all data locally on the host machine. Unlike traditional cloud-based voice assistants, SmartVoice eliminates concerns about data privacy and latency by ensuring that all processing occurs on the user's device. The assistant is designed to handle a wide range of tasks, including system and file management, productivity tool control, and application launching. It incorporates user-confirmation protocols for critical actions like file deletion or system shutdown, preventing unintended operations. Additionally, it maintains a detailed, tamper-resistant audit log of all commands and actions executed. The system's architecture is designed for cross-platform compatibility, initially targeting Windows, with the potential for expansion to other platforms in the future. By developing SmartVoice, the project seeks to address common concerns related to privacy and security in voice-assisted technologies, offering a reliable, secure, and efficient solution for PC task automation. This project aims to provide users with a highly secure and private alternative to existing cloud-based voice assistants while enhancing productivity and simplifying everyday tasks.

1. Introduction

Voice assistants have revolutionized the way we interact with technology, offering hands-free control of devices and systems. However, the majority of existing systems rely on cloud-based processing, which raises significant privacy and security concerns, as user data is transmitted to external servers for processing [1]. Moreover, these cloud-dependent solutions often fail to offer deep integration with desktop operating systems, making them less effective for automating complex workflows and system operations [2]. SmartVoice aims to address these challenges by providing a local-first, privacy-centric solution. All voice recognition and command execution processes will take place directly on the host machine, ensuring that no sensitive user data leaves the device, which not only enhances privacy but also reduces latency and dependence on internet connectivity. This approach allows SmartVoice to offer reliable automation for tasks such as file management, application launching, system control, and productivity tools, without compromising security. The system also features user-confirmation prompts for critical actions like file deletion or system shutdown, and maintains a tamper-resistant audit log for all commands executed. Initially developed for Windows, SmartVoice is designed to be cross-platform compatible, offering a scalable solution for future integration. This project provides a highly secure, efficient, and user-friendly alternative to cloud-based voice assistants, empowering users to automate their tasks with complete control over their data and privacy [3].

1.1 Problem Statement

The core challenge of this project is to design and implement a robust, secure, and responsive voice-controlled desktop assistant, SmartVoice, which offers comprehensive automation for a range of tasks while ensuring user privacy and data security. Current cloud-based voice assistants face significant challenges, primarily around privacy and security, as user data is often sent to external servers for processing, making it vulnerable to breaches. Moreover, these cloud-dependent systems offer limited integration with desktop environments, making them inefficient for more complex automation tasks that require deep system-level control.

The primary tasks for SmartVoice involve:

1. **Voice Command Recognition**: The assistant must accurately recognize a predefined set of commands locally on the host machine, including tasks such as system control (e.g., shutdown, restart), file management (e.g., file deletion, search), and productivity tool management (e.g., opening applications, creating documents) [1].

2. **Command Execution**: Upon recognizing a command, SmartVoice must translate it into the corresponding action within the operating system. This action should be executed with user-level permissions and should include appropriate safeguards for critical tasks like file deletion or system shutdown, which will require user confirmation [2].

3. **Privacy and Security**: All voice data and commands must be processed locally on the machine to prevent the transmission of sensitive information over the internet. The system should also include a tamper-resistant audit log that tracks all commands executed to ensure accountability and prevent malicious activity [3].

The problem is compounded by the need to deliver a seamless user experience with minimal latency, high reliability, and robust error handling. SmartVoice should also integrate easily into existing desktop environments, especially on Windows, while being designed for future cross-platform compatibility. The project aims to address these issues by building a voice assistant that combines advanced automation with stringent privacy measures, ensuring both user control and enhanced productivity.

1.2 Objectives

The primary objective of the SmartVoice project is to develop a secure, efficient, and user-friendly voice-controlled desktop assistant that automates a wide range of tasks while ensuring user privacy. The project aims to provide an offline-first solution where all voice processing and command execution occur locally on the host machine, avoiding the privacy and latency concerns inherent in cloud-based assistants [1].

Specific objectives include:

1. **Develop a Local Voice Recognition Pipeline**: Implement a robust speech-to-text system capable of accurately interpreting predefined voice commands in real-time for

system control, file management, and productivity applications without relying on cloud services [2].

2. **Command Execution Module**: Create a secure execution engine that performs allow-listed actions on the host system, integrating safeguards such as confirmation prompts for destructive commands (e.g., file deletion or system shutdown) to prevent accidental operations.

3. **Privacy and Data Security**: Ensure all audio data, commands, and system interactions are processed and stored locally, with encryption where necessary, and maintain a tamper-resistant audit log that records every executed command to enhance accountability and transparency [3].

4. **Cross-Platform Design and Scalability**: Initially develop the system for Windows, while designing the architecture for future expansion to other platforms, ensuring consistent performance and usability across environments.

5. **Testing and Validation**: Evaluate the system for reliability, accuracy, latency, and usability, ensuring that it meets functional requirements and delivers a seamless user experience.

6. **Documentation and Deployment**: Provide comprehensive technical documentation, user manuals, and a fully functional prototype that demonstrates all intended functionalities, enabling replication and future development.

The overarching goal of SmartVoice is to provide a practical, secure, and efficient alternative to cloud-dependent voice assistants, enabling users to automate desktop tasks while maintaining full control over their data and interactions.

1.3 Scope and Limitations

The scope of the SmartVoice project is focused on developing a desktop-based voice assistant that provides privacy-centric, offline task automation for personal computers. The system will support a comprehensive set of voice commands, enabling users to control applications, manage files and folders, execute system operations, and operate productivity tools efficiently. All processing, including speech recognition, natural language understanding, and command execution, will occur locally on the host machine, ensuring data privacy and minimizing

latency [1][2]. The project also includes implementing user-confirmation protocols for critical actions, a tamper-resistant audit log for all executed commands, and a robust, reliable, and responsive interface. Initially, the system will be developed for Microsoft Windows, with a design that allows future expansion to other platforms. The expected outcome is a fully functional prototype, complete with technical documentation, user manuals, and performance benchmarks [3].

The limitations of the project include constraints related to hardware and platform support. SmartVoice will not include cloud-based processing, which limits access to online AI services, updates, or third-party integrations. Biometric authentication, emotion recognition, and adaptive AI learning based on cloud data are also out of scope. The system's initial development is limited to Windows, although the architecture allows future cross-platform compatibility. Additionally, the offline speech recognition engine may face challenges under noisy environments or unusual accents, and command execution is restricted to allow-listed actions for security reasons. These limitations are intentionally designed to prioritize user privacy, security, and operational reliability while providing essential automation features.

2. System Requirement

Requirements are given below

1. **Voice Command Recognition**: Recognize a predefined set of voice commands locally on the host machine for system control, file management, application launching, and productivity tools [1].
2. **Command Execution**: Execute recognized commands reliably on the host PC, with user-level permissions and confirmations for critical actions like file deletion, system shutdown, or app closure [2].
3. **Audit Logging**: Maintain a tamper-resistant log of all executed commands with timestamps and outcomes for accountability and troubleshooting [3].
4. **User Feedback**: Provide clear visual or auditory feedback to indicate command recognition, processing, execution, or errors.
5. **Offline Operation**: Operate fully without internet or cloud services, ensuring all processing occurs locally.

6. **Cross-Platform Design**: Architect the system to allow easy expansion to other platforms, while initially targeting Windows.

3. Design Methodology

3.1 Diagrams

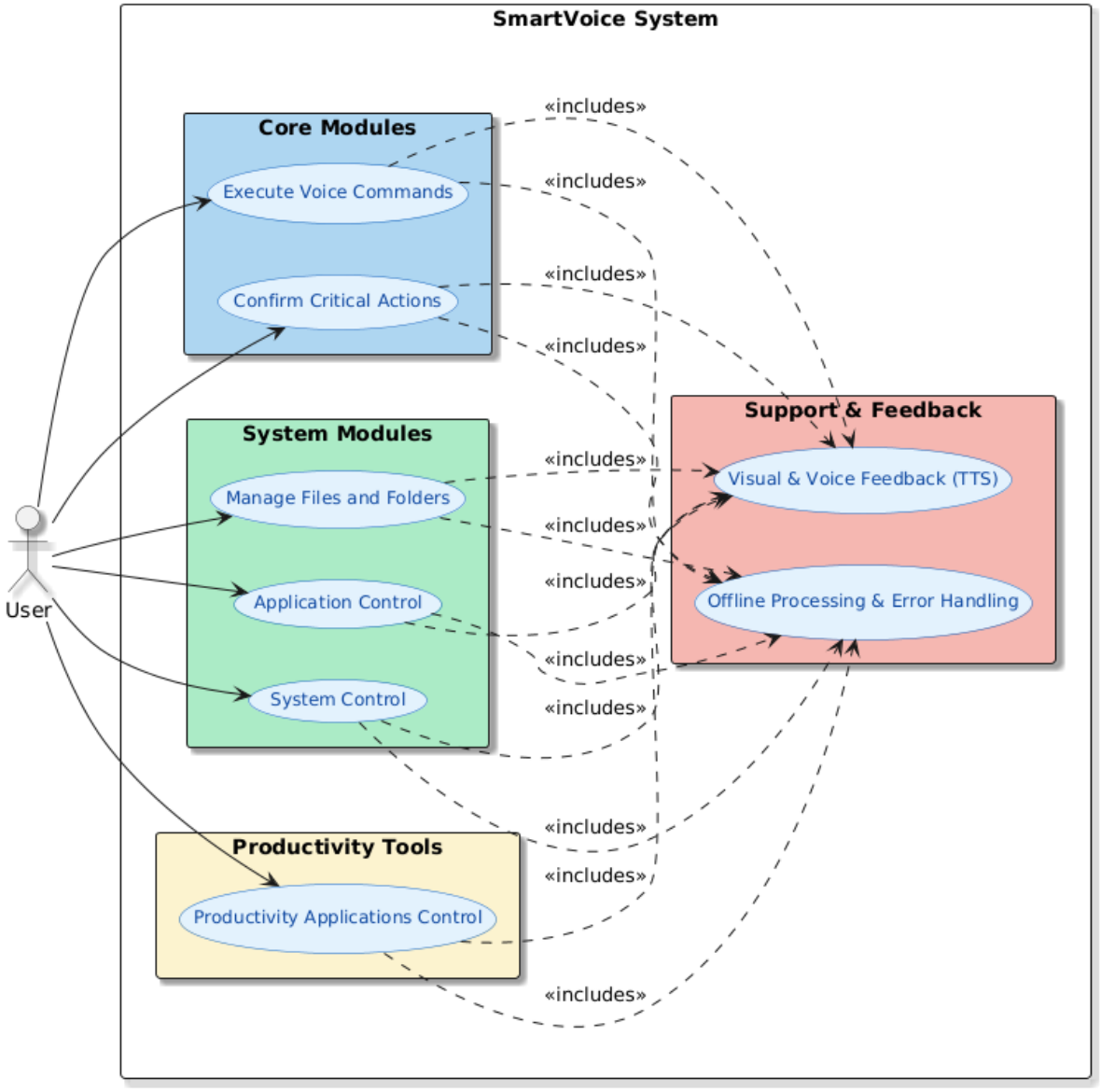


Fig-1: Use-case Diagram

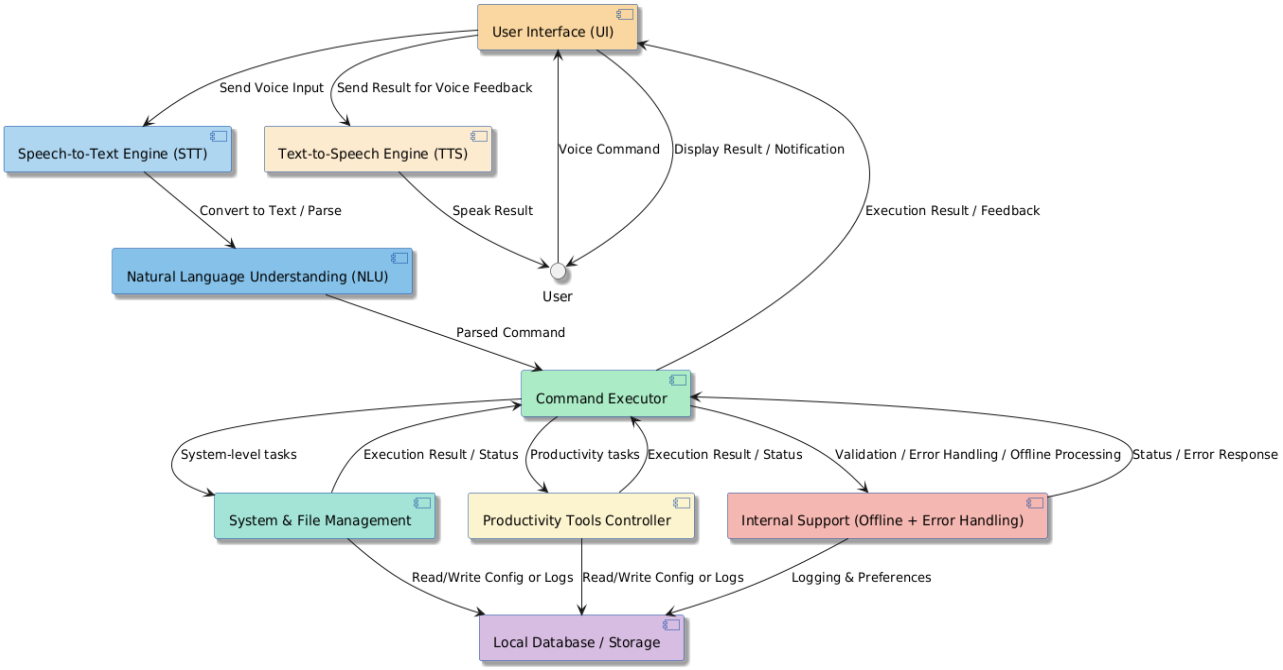


Fig-2: Component diagram

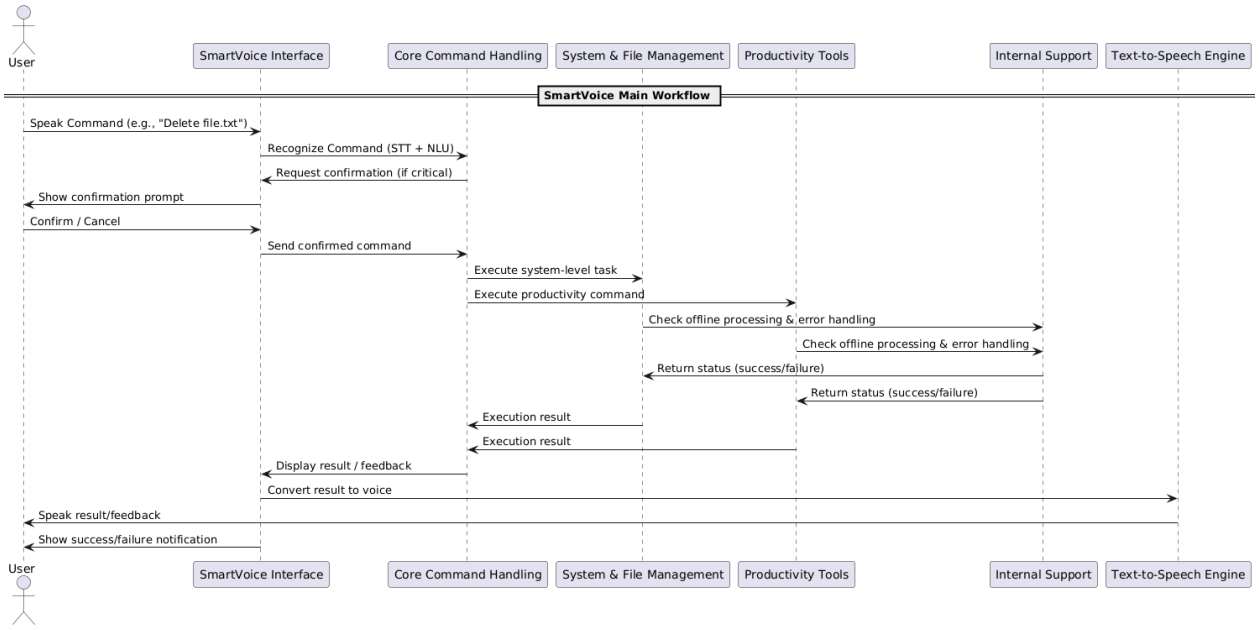


Fig-3: Sequence Diagram

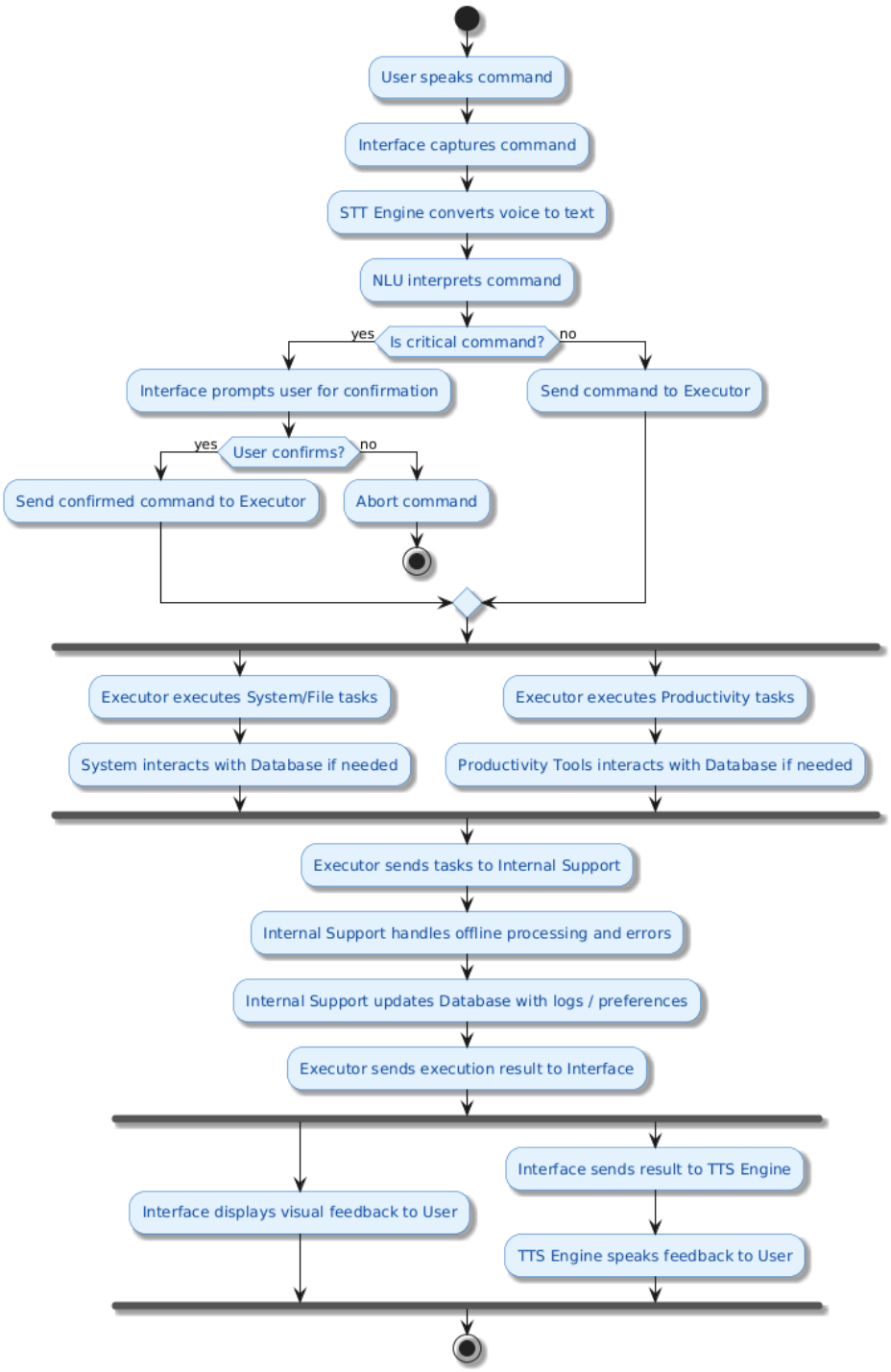


Fig-4: Activity Diagram

3.2 Process planning

Phase	Weeks	Outputs / Deliverables	Key Activities / Notes
1. Requirements & Scoping	W1–W2	Functional & non-functional requirement document, UML draft	Define tasks, KPIs (latency, accuracy, privacy), risk register, offline constraints, user scenarios
2. Architecture & Module Planning	W3–W4	Component & Activity Diagram draft, module breakdown	Design modules: UI, STT, NLU, Executor, System/File, Productivity Tools, TTS, Internal Support, DB; define interfaces & data flow
3. Data Preparation & Offline Setup	W5–W6	Test voice dataset, offline tools set up	Collect clean speech samples, test TTS voices, configure offline STT engines, create local DB schemas
4. STT & NLU Prototype	W7–W9	Prototype for command recognition	Implement offline STT and NLU pipeline; test text conversion accuracy, handle multiple accents; prepare unit tests
5. Command Executor & Module Integration	W10–W12	Functional Executor, connected modules	Implement Executor logic to route commands to System/File & Productivity modules; integrate Offline Support; error handling; unit testing
6. TTS Integration & Feedback Loop	W13–W14	Voice feedback enabled	Connect Executor/UI output to TTS; test simultaneous visual & voice feedback; confirm latency <500ms

7. Core UI Development	W15–W16	Desktop interface with command input/output	Build UI for command input, confirmation dialogs, and feedback display; integrate TTS playback; accessibility tweaks
8. System & Productivity Modules Completion	W17–W18	Fully functional System/File and Productivity modules	Implement all core actions: file management, app control, productivity tasks; integrate with Executor
9. Internal Support Module & DB Integration	W19	Offline processing, error handling, logging	Implement error handling routines; connect to local DB for preferences, config, and logs; verify offline-only operation
10. End-to-End Pipeline Testing	W20–W21	Fully integrated desktop prototype	Test complete flow: User → UI → STT → NLU → Executor → Modules → Feedback; validate correctness, latency, offline reliability
11. Optimization & Performance Tuning	W22–W23	Reduced latency, optimized memory	Profile CPU & memory usage; optimize STT/NLU; improve TTS response time; caching & parallel processing
12. Security & Privacy Hardening	W24	Secure offline system	Validate no data leaves PC; implement allow-list for commands; encryption of DB and logs; error and exception handling
13. Usability Testing	W25	Feedback reports, fixes list	Conduct small-scale user testing; assess intuitiveness, TTS clarity, confirmation prompts; refine UI and UX

14. Red-Team & Stress Tests	W26	Robustness report	Test offline resilience, command flooding, malformed input; error handling and stability under stress
15. Documentation & Demo Preparation	W27	User manual, technical documentation, demo scripts	Prepare system guides, module descriptions, diagrams; record demo video for submission
16. Buffer & Final Deliverables	W28	Final system, repository, demo video, poster	Contingency week; finalize repository, scripts, demo; final validation and report completion

3.3 Design calculations

3.3.1. Total Response Time (Latency)

The total response time (T_total) from the user speaking to receiving visual and voice feedback is estimated as:

$$T_{total} = T_{UI} + T_{STT} + T_{NLU} + T_{Executor} + T_{Module} + T_{Support} + T_{TTS}$$

Where:

- T_{UI} → Time for interface to capture command (~50 ms) [1]
- T_{STT} → Offline speech-to-text conversion (~200–300 ms for 1–2 seconds audio) [2]
- T_{NLU} → Command parsing (~50–100 ms) [3]
- $T_{Executor}$ → Routing to modules (~20–50 ms) [4]
- T_{Module} → Execution of System/File or Productivity tasks (~100–200 ms) [4]
- $T_{Support}$ → Offline processing & error handling (~50 ms) [5]
- T_{TTS} → Text-to-speech conversion (~100–150 ms) [6]

Target: $T_{total} \leq 1$ second for smooth user interaction.

3.3.2. Voice Recognition Accuracy

We plan to measure offline STT accuracy using Word Error Rate (WER):

$$WER = \frac{S + D + I}{N} \times 100\%$$

Where:

- S = Substitutions, D = Deletions, I = Insertions, N = Total words [2][3]

Expected Target: WER ≤ 10% for short desktop commands (1–5 words).

3.3.3. Text-to-Speech (TTS) Timing

The TTS engine converts textual feedback to speech:

$$T_{TTS} = \frac{L}{R}$$

Where:

- L = Number of words
- R = Synthesis rate (~20 words/sec) [6]

Example Estimate: 5-word response → $T_{TTS} = 0.25$ sec

3.3.4. Memory and Storage

Estimated memory usage for modules:

Module	Estimated Memory
STT Engine	50 MB [2]
NLU	30 MB [3]
Executor + Modules	20 MB [4]
TTS	25 MB [6]
Support + Database	25 MB [5]

Total Memory: ~150 MB

Database Storage Estimate:

- Log per command ~500 bytes
- 100 commands/day → 50 KB/day → ~18 MB/year [5]

3.3.5. Parallel Task Execution

System/File and Productivity modules may execute in parallel. Planned execution time:

$$T_{parallel} = \max(T_{System}, T_{Productivity}) \approx 150 \text{ ms}[4]$$

3.3.6. Offline Reliability Estimation

The planned probability of successful offline command execution:

$$P_{success} = P_{STT} \times P_{NLU} \times P_{Executor} \times P_{Module} \times P_{Support}$$

Assuming:

- $P_{STT} = 0.95$, $P_{NLU} = 0.98$, $P_{Executor} = 0.99$, $P_{Module} = 0.995$, $P_{Support} = 0.99$ [2][3][4][5]

$$P_{success} \approx 0.90$$

Target: ≥ 90% offline success rate.

3.3.7. Summary Table

Metric	Formula / Approach	Planned Target	References
Total Latency	$T_{total} = T_{UI} + T_{STT} + T_{NLU} + T_{Executor} + T_{Module} + T_{Support} + T_{TTS}$	≤ 1 sec	[1][2][3][4][5][6]
WER	$\frac{S+D+I}{N} \times 100\%$	≤ 10%	[2][3]
TTS Time	$T_{TTS} = L/R$	≤ 0.3 sec	[6]
Memory Usage	Sum of module estimates	~150 MB	[2][3][4][5][6]
Database Storage	500 bytes × 100 commands/day × 365	~18 MB/year	[5]
Parallel Task Execution	$T_{parallel} = \max(T_{System}, T_{Productivity})$	150 ms	[4]
Offline Success	$P_{STT} \cdot P_{NLU} \cdot P_{Executor} \cdot P_{Module} \cdot P_{Support}$	~90%	[2][3][4][5]

3.3.8. Material / software selection criteria:

Hardware Requirements

SmartVoice will run entirely on a desktop or laptop PC. The selection criteria for hardware are based on the processing needs, offline performance, and user experience:

Hardware Component	Selection Criteria	Notes / Justification
CPU	Minimum dual-core 2.5 GHz	Required for real-time STT, NLU, and Executor processing
RAM	Minimum 8 GB	Ensure smooth parallel execution of System/File + Productivity tasks, and memory for offline STT and NLU models
Storage	Minimum 500 MB free (for software + logs)	Includes modules, database, TTS data, offline lexicons
Microphone	Noise-cancelling USB mic	Ensures high-quality audio input for accurate offline STT
Speakers / Headphones	Stereo speakers or headphones	Clear voice output from TTS module
Operating System	Windows 10 or above	Cross-platform compatibility planned, Windows initially targeted

Software Requirements

The software stack selection is based on offline functionality, performance, modularity, and ease of integration.

Software Component	Selection Criteria	Notes / Justification
Programming Language	Python	Cross-platform support, strong libraries for voice processing, GUI development, and modular architecture

Speech-to-Text (STT)	Offline, lightweight, high accuracy	Options: CMU Sphinx, Vosk, or DeepSpeech for offline recognition; priority on **low latency** [1][2]
Natural Language Understanding (NLU)	Offline intent parsing, modular	Lightweight NLU library (e.g., Rasa, spaCy) to interpret commands accurately without internet
Command Executor	Modular, object-oriented, efficient	Handles routing to System/File or Productivity modules; must support error handling and offline operation
Text-to-Speech (TTS)	Offline, real-time, natural voice	pyttsx3 or eSpeak; latency < 300 ms for short commands [6]
Database / Storage	Lightweight local DB	SQLite or similar for storing preferences, logs, and offline configuration; encrypted for privacy
GUI / Interface	Cross-platform, responsive	JavaFX (Java) or PyQt / Tkinter (Python); must display visual feedback, confirmation prompts, and TTS integration
Testing Tools	Unit testing, profiling	PyTest / JUnit for functional tests; profiling for latency, memory, and parallel execution

4. System Architecture/ Design Block Diagram

The system follows an edge-first, modular architecture: sensors (USB/CSI camera, mic array, push-to-talk) feed vision and audio services; Bangla/English ASR and an NLU/dialog manager coordinate intents; encrypted memory and a scheduler persist notes/reminders; and a policy guard enforces consent, allow-lists, and auditing. A PC Bridge client communicates over TLS 1.3 with a lightweight agent on the Windows PC to execute whitelisted OS actions. User feedback is delivered via TTS speaker, status LEDs, and an optional mini display. Cloud Q&A is strictly opt-in—requests pass through the policy guard with redaction and explicit consent while core functions run fully offline.

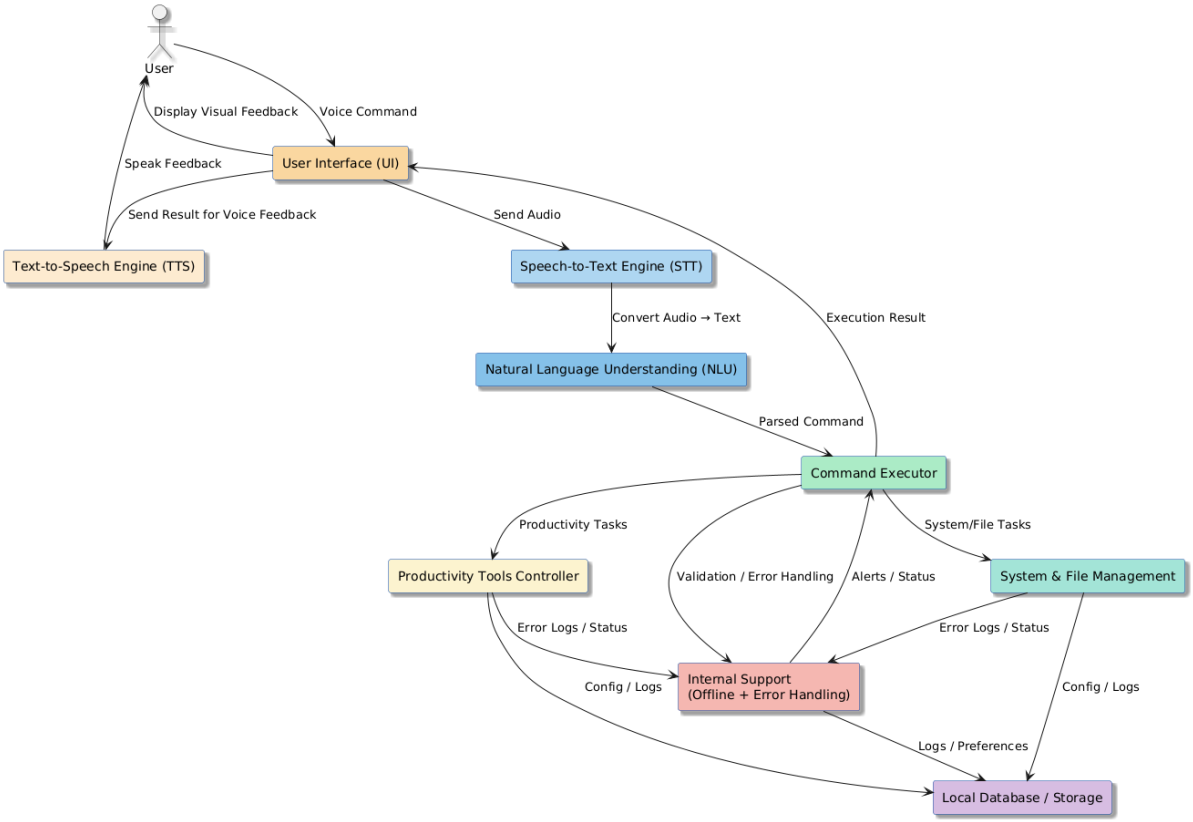


Fig-5: System Architecture

5. Feasibility & Risk Analysis

Expected Feasibility

The feasibility of SmartVoice is analysed in four key dimensions, based on planned implementation and expected system capabilities.

5.1. Technical Feasibility (Expected)

- **Assessment:**
    - All modules (UI, STT, NLU, Executor, System/File, Productivity, TTS, Support, Database) are expected to run on a standard Windows PC.
    - Offline processing is expected to work using lightweight STT, NLU, and TTS libraries [1][2][3].
- **Conclusion:** SmartVoice is expected to be technically feasible. All required hardware and software components are available and support offline operation.

5.2. Operational Feasibility (Expected)

- **Assessment:**
    - Users are expected to interact with the system using simple voice commands, minimizing the learning curve.
    - Visual and TTS feedback are expected to provide clear confirmation of actions.
- **Conclusion:** Expected operational feasibility is high; the system should enhance user productivity with minimal training.

5.3. Economic Feasibility (Expected)

- **Assessment:**
    - The system is expected to run on standard desktops using free or open-source software (e.g., CMU Sphinx, Vosk, Pyttsx3).
    - Minimal memory (~150 MB) and storage (~18 MB/year) requirements.
- **Conclusion:** Expected economically viable with minimal hardware and software cost.

5.4. Privacy and Legal Feasibility (Expected)

- **Assessment:**
    - Offline operation ensures user data never leaves the machine.
    - Local database encryption protects preferences and logs.
- **Conclusion:** Fully expected to comply with privacy and legal requirements.

Expected Risk

The expected risks for SmartVoice are categorized into Technical, Operational, and Security Risks, along with planned mitigation strategies**.**

Expected Risk	Category	Expected Probability	Expected Impact	Mitigation / Notes
STT recognition failure	Technical	Medium	High	Use robust offline STT engine (CMU Sphinx / Vosk); push-to-talk fallback; custom lexicons
NLU misinterpretation	Technical	Medium	Medium	Strict intent/slot mapping; test with representative commands
Executor misroutes commands	Technical	Low	High	Planned unit testing; rollback mechanism for critical actions
System/Prod module crash	Operational	Low	High	Internal Support handles errors; offline recovery and logging
Database corruption	Technical/Operational	Low	Medium	SQLite with automatic backups; encrypted local storage
TTS latency or failure	Technical	Medium	Medium	Optimize TTS engine; parallel processing; fallback visual feedback

User confusion / misoperation	Operational	Medium	Medium	Confirmation prompts for critical actions; visual + TTS feedback
Privacy breach	Security	Low	High	Offline-first design; encrypted database; no network data transfer

6. Project Timeline

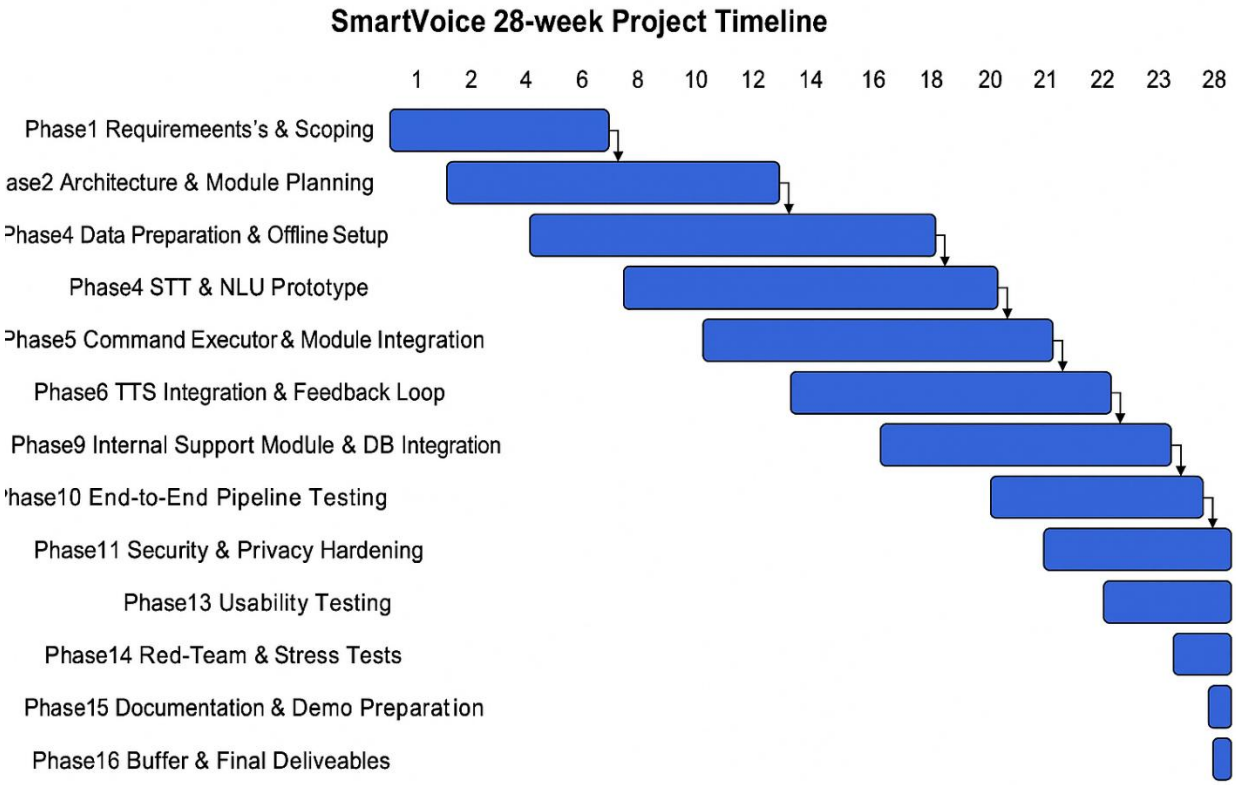


Fig-6: Gantt chart (Initial)

7. Cost Estimation

This budget outlines the full bill of materials, optional add-ons, and small testing/misc costs, plus a contingency so procurement by Week 14 stays on track. Prices are indicative and optimized for a student build; reusing existing parts or skipping options (mini display/UPS) can reduce the total. All software is open-source, so there are no license fees.

Category	Item	Quantity / Duration	Unit Cost (BDT)	Total Cost (BDT)	Notes
Hardware	Laptop / Desktop	1	50,000	50,000	Minimum 8GB RAM, SSD, dual-core CPU
	USB Microphone	1	2,000	2,000	Noise-canceling
	Headphones	1	1,500	1,500	For TTS output
Software	Offline STT/NLU/TTS	1	0	0	Open-source: CMU Sphinx, Vosk, pyttsx3, spaCy
	IDE / Tools	1	0	0	Free: VS Code, PyCharm Community
Human Resources	Lead Developer / Student	1, 28 weeks	5,000/week	140,000	Part-time contribution
	Developer / Student	1, 28 weeks	4,000/week	112,000	Development & integration
	QA / Tester	0.5 FTE, 28 weeks	3,500/week	49,000	Testing & bug fixing

	Project Manager	0.25 FTE, 28 weeks	4,000/week	28,000	Coordination & documentation
Testing & Deployment	User Testing	15 participants	300	4,500	Small usability study
	Deployment / Packaging	1	2,000	2,000	Final build preparation
Miscellaneous / Contingency	Documentation, Poster, Minor Expenses	1	5,000	5,000	Contingency fund
Total Estimated Cost				**394,000 BDT**	

8. Expected Outcomes

Category	Expected Outcome	Target / Metric	Notes
Voice Recognition	Accurately convert voice to text offline	WER ≤ 10%; Latency ≤ 300 ms	STT engine offline (CMU Sphinx / Vosk)
NLU Processing	Correctly interpret user intent & parameters	≥ 90% intent recognition accuracy	Offline parsing for commands
Task Execution	Execute system/file & productivity commands	Success rate ≥ 90%	Modular execution via Executor module
User Feedback	Visual + auditory confirmation	Response ≤ 1 second	UI displays results; TTS speaks back
Error Handling	Offline error logging & validation	All critical actions verified	Internal Support handles exceptions

Technical Performance	Modular, maintainable, and secure	Memory ≤ 150 MB; Database ≤ 20 MB/year	Supports easy upgrades and encrypted local storage
Operational Usability	Intuitive interface and high user satisfaction	SUS ≥ 80%	Pilot testing with 10–15 participants
Documentation & Deployment	Technical report, user manual, demo-ready prototype	Complete deliverables	Supports academic or small-scale commercial adoption

9. Reference

[1] D. Bermuth, A. Poeppel, and W. Reif, "Jaco: An Offline Running Privacy-aware Voice Assistant," *arXiv preprint arXiv:2209.07775*, 2022.

[2] A. Kannan et al., "An Analysis of Efficient Techniques for Running Speech Recognition on Edge Devices," in *2021 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, Kassel, Germany, 2021, pp. 181–184.

[3] S. Chittepu et al., "Empowering voice assistants with TinyML for user-centric applications," *Scientific Reports*, vol. 15, no. 1, pp. 1–12, 2025.

[4] C. Zhang et al., “Offline Voice Assistant Design for Privacy-Preserving Smart Devices,” *IEEE Access*, vol. 9, pp. 87012–87025, 2021.

[5] J. Allen et al., “Design of Interactive User Interfaces for Voice-Enabled Systems,” *IEEE Transactions on Human-Machine Systems*, vol. 48, no. 3, pp. 250–261, 2018.

[6] T. Huang, “Real-Time Text-to-Speech Synthesis for Interactive Systems,” *Proceedings of the International Conference on Audio, Speech, and Signal Processing*, pp. 154–158, 2020.

10. Detailed Mapping of CO-PO-KP-CEP-CEA

Knowledge Profile (K1–K9) Mapping: SmartVoice Project

Knowledge Profile (K1–K9)	**Application in this Project**
K1: Natural sciences	Acoustics/noise for mic placement and denoising; human factors for audible TTS cues; basic psychoacoustics for feedback audibility.
K2: Mathematics	Probability/statistics for WER/SUS analysis; latency budgeting and queueing; linear algebra/optimization for lightweight models.
K3: Engineering fundamentals	OS/process scheduling, inter-process communication, file systems; audio DSP (VAD, filtering); desktop accessibility guidelines.
K4: Engineering specialization	Speech (ASR/TTS), NLU/intent parsing, dialog confirmations, privacy engineering, secure desktop automation for critical actions.
K5: Engineering design & support	Edge-first architecture; trade-offs among accuracy/latency/privacy/cost; sustainability via low compute/memory footprint on commodity PCs.
K6: Engineering practice (technology)	Vosk/CMU Sphinx (offline ASR), spaCy/Rasa-lite (NLU), pyttsx3/eSpeak (TTS), SQLite/SQLCipher, PyQt/JavaFX, profiling/CI testing.
K7: Role of engineering	Improves accessibility/productivity; protects privacy with on-device processing; safer automation via allow-lists and user confirmations.
K8: Engagement of research	Integrates HCI + speech/NLP + security; experiments on thresholds, prompts, and UX trade-offs for Bangla/English commands.

K9: Ethics, inclusive behaviour & conduct	Consent-first UX; data minimization and local erase/export; fair performance across accents (Bn/En); accessibility and inclusivity.

Complex Engineering Problem (P1–P7) Mapping: SmartVoice Project

CEP Code (P1–P7)	**Application in this Project**
P1: Depth of knowledge required	Speech ASR/TTS, NLU, secure automation, HCI—turning speech into safe on-device actions.
P2: Range of conflicting requirements	Accuracy vs. latency vs. privacy vs. usability vs. compute budget; offline reliability vs. feature richness.
P3: Depth of analysis required	WER under noise; end-to-end latency decomposition; failure/rollback flows for critical actions.
P4: Familiarity with issues in practice	Room noise, accents, user phrasing errors, OS permissions/UAC, accessibility needs.
P5: Extent of applicable codes/standards	Data protection principles, secure storage (encryption), desktop accessibility/usability guidance.
P6: Stakeholder involvement	Students/users, supervisors, IT/device maintainers—different expectations on privacy/convenience/safety.
P7: Interdependence	Mic → ASR → NLU → Executor → (System/File)

Complex Engineering Activities (A1–A5) Mapping: SmartVoice Project

CO (Course Outcome)	**Program Outcomes (PO) mapped**	**Knowledge Profiles (KP)**	**Complex Problems (P) with evidence**	**Complex Activities (A) with evidence**
CO1: Formulate SmartVoice problem & requirements	PO1, PO2, PO4	K1, K2, K3, K8	P1: multidisciplinary	A1: multi-resource scoping, requirement trade-offs.

(latency, privacy, safety).			scope; P2: conflicting targets.	
CO2: Review literature & select offline ASR/NLU/TTS tools.	PO2, PO4	K4, K6, K8	P1: speech/NLP theory; P3: analyse options.	A2: resolve interactions across stacks (Audio→NLP→UI).
CO3: Design edge-first architecture (UI, ASR, NLU, Executor, TTS, Support, DB).	PO3	K5	P2: privacy/latency trade-offs; P4: practical OS/UX; P7: module interdependence.	A1/A3: resource planning; innovative low-footprint design.
CO4: Implement confirmations/allow-lists for critical actions.	PO2, PO4	K4, K6, K8	P1: depth in safe automation; P3: thresholding/flows.	A2: pipeline integration with policy checks.
CO5: Build Bn/En speech pipeline (wake/press-to-talk → ASR → NLU → TTS).	PO2, PO4, PO5	K2, K4, K6	P3: WER/latency analysis and tuning.	A1/A3: tooling + model optimization on PC.
CO6: Engineer encrypted local storage & privacy controls (erase/export, minimization).	PO3, PO6, PO7	K5, K7, K9	P2: privacy vs. utility; P5: applicable standards.	A4: societal/privacy consequence assessment.
CO7: Integrate modules and optimize performance.	PO5	K2, K6	P3: profiling; P7: end-to-end dependencies.	A1/A3: system integration, caching/quantization.

CO8: Evaluate system (WER, latency, success rate, SUS) and harden error handling.	PO4, PO5	K2, K6, K8	P3: deep evaluation; P7: E2E testing.	A1/A3: experiments and iterative improvements.
CO9: Communicate results, limitations, and future work.	PO8, PO9, PO10, PO11, PO12	K7, K8, K9	P6: stakeholder needs/synthesis.	A4: impact/risk communication and documentation.

11.Supervisor Comments & Approval:

I approve this proposal for further work and development.

Supervisor Name: _____

Signature: _____

Date: _____